1.INTRODUCTION

1. INTRODUCTION

1.1 PROJECT SCOPE

The Missing Child Identification System is a user-friendly application designed to assist in the quick and efficient management of information about missing children. This system enables users to add detailed records of missing children and search for these records based on the child's name. By leveraging simple yet effective data storage techniques, the system aims to streamline the process of recording, updating, and retrieving vital information, thereby aiding law enforcement agencies, social workers, and concerned citizens in their efforts to locate and reunite missing children with their families.

The main goals of the Missing Child Identification System are to:

1. Centralize Data: Provide a unified platform where information about missing children can be stored securely and accessed easily.

2. Enhance Search Efficiency: Allow quick retrieval of information through a simple search functionality, reducing the time taken to find critical details.

3. Ensure Data Accuracy: Maintain accurate and up-to-date records by allowing users to input comprehensive details about each missing child.

4.Facilitate Coordination: Improve coordination between different stakeholders involved in the search for missing children by providing a common database that can be accessed by authorized personnel.

5. Improve Accessibility: Ensure that the system is accessible to users with varying levels of technical expertise through a straightforward command-line interface.

PROJECT FEATURES

In India a countless number of children are reported missing every year. Among the missing child cases a large percentage of children remain untraced. Use of deep learning methodology for identifying the reported missing child from the photos of multitude of children available, with the help of Names and remarks etc.

Upload photographs of suspicious child into a common portal with landmarks and remarks. The name of the missed child will be automatically compared with the registered names of the missing child from the repository. For this, a deep learning model is trained to correctly identify the missing child from the missing child image database provided, using the facial image uploaded by the public.

2.SYSTEM ANALYSIS

2. SYSTEM ANALYSIS

2.1 Analysis

The analysis and design phase is critical in ensuring that the Missing Child Identification System meets the needs of its users effectively. This phase involved gathering detailed requirements, designing the system architecture, and planning the implementation strategy. The requirements were gathered through interviews with potential users, including law enforcement officials, social workers, and family members of missing children. This collaborative approach ensured that the system addresses real-world needs and challenges.

1. User-Friendly Interface:

- Ease of Use: The system must be intuitive and easy to navigate, allowing users to quickly input and retrieve information without extensive training. This is crucial for ensuring that even non-technical users, such as family members and volunteers, can use the system effectively.
- Accessibility: The interface should be accessible to users with varying levels of technical expertise. This includes considerations for clear instructions, minimalistic design, and support for different languages and accessibility options for users with disabilities.

2. Comprehensive Data Fields:

- Essential Information: The system must capture all critical information about missing children. This includes the child's name, age, gender, height, weight, hair color, eye color, and a photo. Additional fields may include the date of disappearance, last known location, and any distinguishing marks or medical conditions.
- Structured Data Entry: The input forms should guide users to enter data in a structured manner, reducing the likelihood of errors. This can be achieved through dropdown menus, radio buttons, and validation checks.

3. Secure Data Storage:

- Data Privacy: Protecting the privacy of missing children and their families is paramount. The system must implement robust security measures to prevent unauthorized access to sensitive information.
- Encryption: All data should be encrypted both in transit and at rest to protect it from potential breaches.
- Access Control: Implement role-based access control to ensure that only authorized personnel can access certain types of information. For example, detailed case notes might be accessible only to law enforcement officials, while general information could be available to

the public.

4. Search Functionality:

- Multiple Search Criteria: Users must be able to search for missing children based on various criteria such as name, age, gender, physical description, and location. This flexibility allows for more targeted and efficient searches.
- Advanced Filtering: Advanced search options should be available to narrow down search results. This could include filters for recent sightings, specific regions, or matching certain physical characteristics.
- Real-Time Updates: Search results should reflect the most current data available, ensuring that users have access to the latest information.

System Design

Based on the gathered requirements, the system was designed with the following components and features:

1. User Interface Design:

- Main Dashboard: The main dashboard provides quick access to key functions such as adding a new missing child, searching for children, and viewing recently added or updated cases.
- Input Forms: User-friendly forms for entering information about missing children. These forms include fields for all essential data and provide clear guidance on required and optional fields.
- Search Interface: A dedicated search interface allows users to enter multiple search criteria and view search results in a clear, organized manner. The search results page includes filtering and sorting options.

2. Database Design:

- Data Structure: The database is designed to store comprehensive information about each missing child in a structured format. Tables include fields for personal details, physical description, and photos.
- Relationships: The database supports relationships between different tables, such as linking sighting reports to specific missing child cases. This allows for a more detailed and interconnected data model.

3. Security Measures:

- Authentication: Implement user authentication to ensure that only authorized users can

access the system. This includes multi-factor authentication for added security.

- Data Encryption: Encrypt all data stored in the database and during transmission over networks. Use industry-standard encryption protocols such as AES-256 for data at rest and TLS for data in transit.

- Access Controls: Role-based access control (RBAC) restricts access to sensitive information based on user roles. For instance, law enforcement officers have access to all data, while public users can only view non-sensitive information.

4. Search and Retrieval System:

- Search Algorithm: Develop an efficient search algorithm that can handle multiple criteria and return accurate results quickly. The algorithm should support partial matches and fuzzy searches to accommodate potential data entry errors.

- Indexing: Implement indexing on key fields such as name, age, and location to speed up search queries.

- Real-Time Updates: Ensure that the database is updated in real-time, so that search results reflect the latest information.

Implementation Strategy

The implementation strategy outlines the step-by-step process for developing and deploying the system:

1. Prototype Development:

- Initial Prototype: Develop an initial prototype of the user interface and basic functionality, such as adding and searching for missing children. This prototype will be used to gather feedback from stakeholders and refine the design.

- User Testing: Conduct user testing sessions with potential users to identify usability issues and gather suggestions for improvement.

2. Core Development:

- Database Implementation: Set up the database structure and implement data encryption and access control measures.

- Backend Development: Develop the backend logic for handling data input, storage, and retrieval. This includes implementing the search algorithm and ensuring real-time updates.

- Frontend Development: Complete the development of the user interface, incorporating feedback from the prototype testing phase.

3. Integration and Testing:

- System Integration: Integrate all components of the system and ensure they work together seamlessly.
- Security Testing: Conduct security testing to identify and address potential vulnerabilities.
- Performance Testing: Test the system's performance under various conditions to ensure it can handle the expected load and provide fast response times.

4. Deployment and Training:

- Deployment: Deploy the system to a live environment, ensuring that all security measures are in place.
- User Training: Provide training sessions and documentation to help users understand how to use the system effectively.

5. Maintenance and Updates:

- Ongoing Support: Provide ongoing support to address any issues that arise and to incorporate user feedback.
- Regular Updates: Regularly update the system to add new features, improve performance, and maintain security.

By following this detailed analysis and design approach, the Missing Child Identification System aims to provide a comprehensive, user-friendly, and secure solution for managing information about missing children, ultimately aiding in their quicker recovery.

2.2 EXISTING SYSTEM

- Mostly missing child cases are reported to the police. For various reasons, the child missing from one region may be found in another region or another state.
- So even if a child is found, it is difficult to identify them from the reported missing cases. No Framework to upload and provide child adoption.
- Oral search by asking people where it takes lot of time in finding the missing child identification
- This existing system are already using in places like
  - Police stations
  - Child Welfare Organizations
  - Child Protection Agencies

2.2.1 LIMITATIONS OF EXISTING SYSTEM

False Positives: Facial recognition algorithms may produce false positives, leading to incorrect matches and potential distress to families involved.

Technological Limitations: Reliance on technology, such as facial recognition, may face limitations in accuracy, particularly in cases of outdated or poor-quality images.

Cost and Resources: Developing and maintaining a robust missing child identification system requires significant financial and human resources, which may pose challenges, especially for smaller organizations or developing countries.

Accessibility Issues: Not all individuals may have access to the internet or smartphones, limiting their ability to report or search for missing children through the system.

2.3 PROPOSED SYSTEM

A new machine learning framework has been developed to aid in the search for missing children. This proposed system is designed to make it easy for anyone to access missing child details through any portal at any time. The portal is user-friendly, ensuring that even those with limited technical skills can utilize it effectively.

` The framework provides comprehensive details about missing children, including their name, age, gender, hair color, eye color, weight (in both centimeters and kilograms), and a photo. This detailed information is crucial for enhancing the chances of locating and identifying missing children promptly and accurately.

2.3.1 ADVANTAGES OF THE PROPOSED SYSTEM

- Comparatively an easy, inexpensive, and reliable method compared to other offline systems.
- Improved Accuracy: Utilizing advanced technologies such as identifying missing children by entering name of the children.
- Rapid Identification: In rapid Identification we can find the output image from the given input image in the portal and also we get the all details of the children.

2.4 HARDWARE & SOFTWARE REQUIREMENTS

2.4.1 HARDWARE REQUIREMENTS

Hardware interfaces specify the logical characteristics of each interface between the software product and the hardware components of the system. The following

are some hardware requirements.

- Processor :I5 Processor
- Ram : 4GB
- Hard Disk :512GB

2.4.2 SOFTWARE REQUIREMENTS:

Software Requirements specifies the logical characteristics of each interface and software components of the system. The following are some software requirements

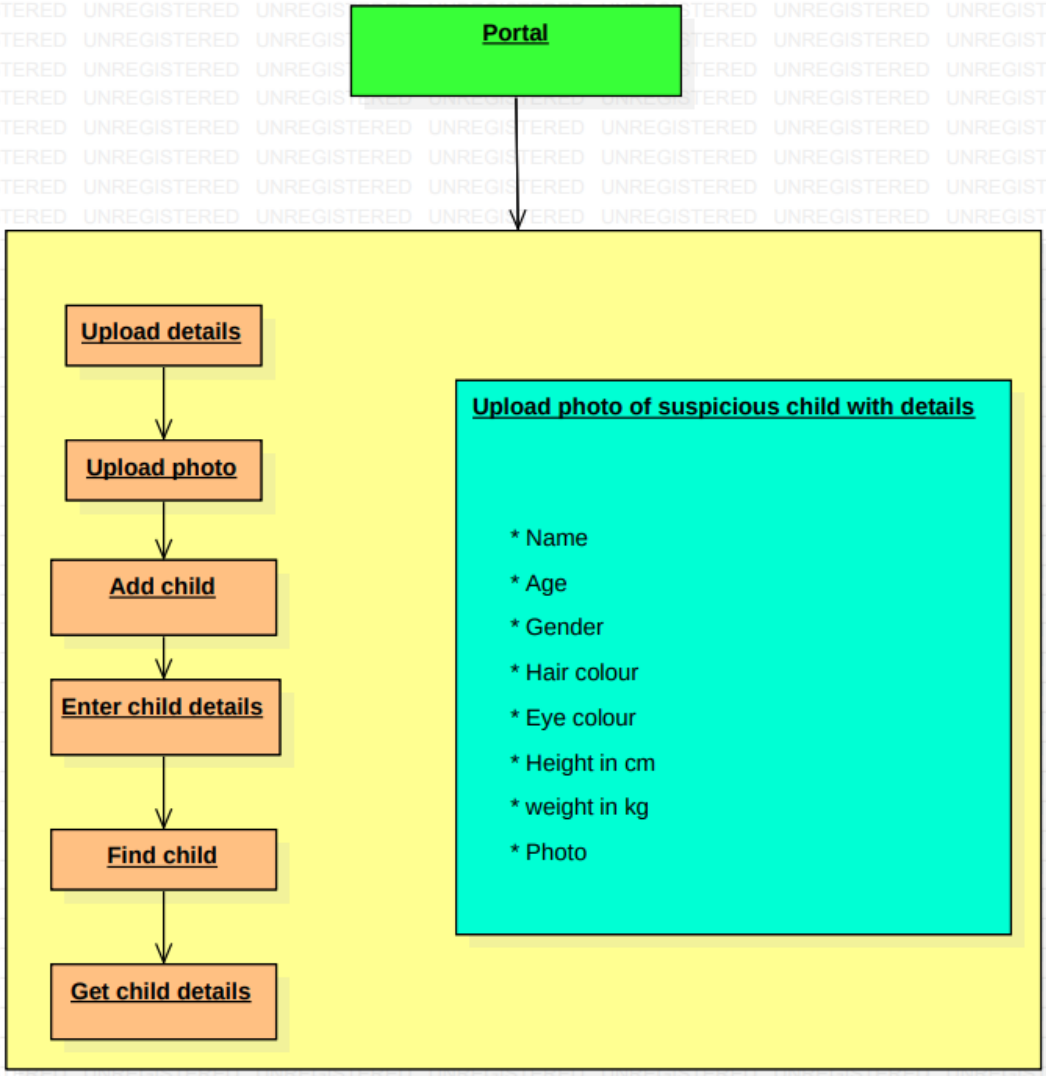
- Operating System :Windows 11
- Technology :Python 3.6
- IDE :VS CODE
- Front-End :HTML
- Back-End : MySQL
- DESIGNING : HTML, CSS, JavaScript

3. ARCHITECTURE

3.ARCHITECTURE

3.1 PROJECT ARCHITECTURE

This project architecture shows the procedure followed for classification, starting from input to final prediction



DESCRIPTION

Proposed Approach

The proposed approach for the new machine learning (ML) framework designed to search for missing children involves several key components and methodologies aimed at maximizing the efficiency and accuracy of locating missing individuals. This comprehensive approach includes data collection, machine learning model development, user interface design, and system integration.

Data Collection and Preprocessing

1. Comprehensive Data Gathering: The framework begins by aggregating data from various sources such as law enforcement databases, missing child organizations, social media platforms, and public records. This data includes personal details, physical attributes, last known whereabouts, and any other relevant information.

2. Data Cleaning and Standardization: The collected data is then cleaned to remove any inconsistencies, duplicates, and inaccuracies. Standardization processes ensure that all data points are in a uniform format, which is crucial for the effective functioning of the ML models.

Machine Learning Model Development

1. Feature Engineering: Key features such as name, age, gender, hair color, eye color, weight, height, and photo are extracted and used to create comprehensive profiles of missing children. Additional contextual features, such as last known location and time, may also be included.

2. Model Training: Advanced ML algorithms, including deep learning models like convolutional neural networks (CNNs) for image recognition and natural language processing (NLP) models for text data, are trained on the collected dataset. These models learn to identify patterns and make predictions about potential matches between missing children and sightings or other relevant data points.

3. Model Evaluation and Optimization: The models are rigorously tested and evaluated using validation datasets to ensure high accuracy and reliability. Continuous optimization techniques are applied to refine the models and improve their performance over time.

User Interface Design

1. Intuitive User Interface: A user-friendly portal is designed to ensure easy navigation and accessibility for users with varying technical expertise. The interface allows users to input search queries and retrieve detailed information about missing children efficiently.

2. Advanced Search Capabilities: The portal includes advanced search functionalities, enabling users to filter and sort results based on different criteria such as age, gender, location, and physical attributes. This helps users narrow down search results quickly and effectively.

System Integration and Scalability

1. Seamless Integration with Existing Systems: The framework is designed to integrate seamlessly with existing databases and systems used by law enforcement agencies, NGOs, and other relevant organizations. This ensures a smooth flow of information and enhances collaboration.

2. Scalability and Adaptability: The system is built to be scalable, accommodating increasing volumes of data and users. It is also adaptable, allowing for the incorporation of new data sources and the evolution of ML models to keep up with changing trends and technologies.

Security and Privacy

1. Data Security Measures: Robust security measures are implemented to protect sensitive data. This includes encryption, secure access controls, and regular security audits.

2. Privacy Protection: The framework adheres to strict privacy regulations and guidelines to ensure the protection of personal information. User data is handled with the utmost confidentiality and used solely for the purpose of locating missing children.

Continuous Improvement

1. Feedback Loop: A feedback mechanism is established to gather input from users and stakeholders. This feedback is used to continuously improve the system's functionality and user experience.

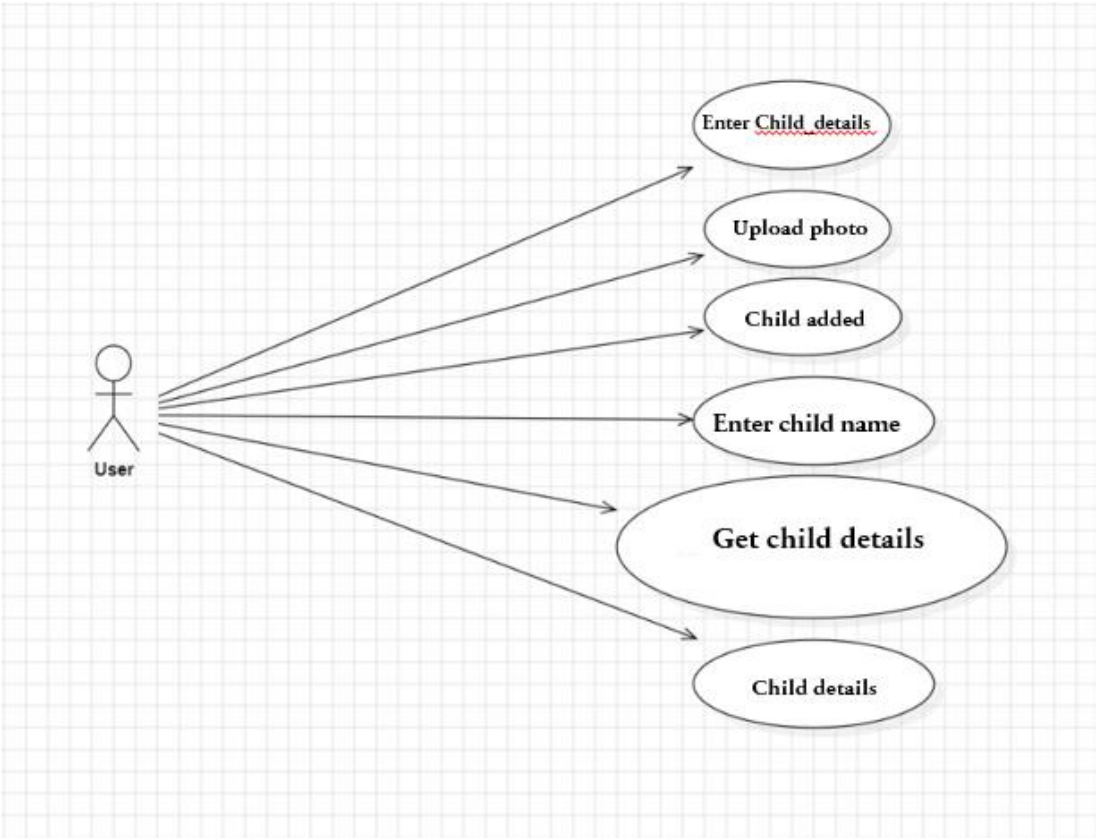
2. Ongoing Research and Development: Continuous research is conducted to explore new technologies and methodologies that can enhance the framework's effectiveness.

This includes staying updated with advancements in ML and AI to incorporate the latest innovations into the system.

By implementing this proposed approach, the ML framework aims to create a powerful, efficient, and user-friendly tool that significantly improves the search for missing children, ultimately increasing the chances of reuniting them with their families.

3.2 USE CASE DIAGRAM

The use case diagram visually represents the interactions between the user and the system, outlining the steps involved in each use case.



DESCRIPTION

The use case diagram for the Missing Child Management System illustrates the various interactions between the system's users (actors) and its functional components (use cases). The key actors in this system are typically law enforcement personnel, NGO

workers, and possibly members of the public who are involved in the process of reporting and searching for missing children. Below is a detailed description of each use case:

Actors

1. User

- The primary user who manages the system. This could be a law enforcement officer, an NGO worker, or a public user with access permissions.

- Responsibilities include uploading child details, managing records, and performing searches.

2. System

- Represents the automated functionalities and services provided by the system.

Use Cases

1. Upload Child Details

- **Description:** The user inputs detailed information about a missing child. This includes the child's name, age, gender, hair color, eye color, and weight.

- **Actors Involved:** User.

- **Steps:**

  1. User logs into the system.

  2. User navigates to the "Upload Child Details" section.

  3. User enters the required information in the provided fields.

  4. User submits the details for storage in the database.

2. Upload Photo

- **Description:** The user uploads a recent photograph of the missing child.

- **Actors Involved:** Admin/User.

- **Steps:**

1. User logs into the system.

2. User navigates to the "Upload Photo" section.

3. User selects a photo file from the local storage.

4. User uploads the photo, which is then linked with the corresponding child details.

3. Add Child Successful

- **Description:** Confirms the successful addition of a child's details and photo into the system.

- **Actors Involved:** System.

- **Steps:**

1. System verifies the completeness and validity of the submitted details and photo.

2. System stores the information in the database.

3. System displays a confirmation message to the user.

4. Find Child by Name

- **Description:** Allows the user to search for a missing child using their name.

- **Actors Involved:** Admin/User.

- **Steps:**

1. User logs into the system.

2. User navigates to the "Find Child by Name" section.

3. User enters the child's name into the search field.

4. User initiates the search.

5. System searches the database for matches.

5. Get Child Details

- **Description:** Retrieves and displays the details of a specific child based on the search criteria.

- **Actors Involved:** System.

- **Steps:**

1. System processes the search query.

2. System retrieves matching records from the database.

3. System displays the child's details, including name, age, gender, hair color, eye color, weight, and the uploaded photo.

Result

Upon performing a successful search, the system will display the following details for the matched child:

- **Name**: The full name of the missing child.

- **Age:** The age of the missing child.

- **Gender**: The gender of the missing child.

- **Hair Color**: The color of the child's hair.

- **Eye Color:** The color of the child's eyes.

- **Weight:** The weight of the child in both centimeters and kilograms.

- **Photo:** The uploaded photograph of the missing child.

3.3 CLASS DIAGRAM

A class diagram provides a static view of the system, showing the system's classes, their attributes, methods, and the relationships among objects. Below is a description of the class diagram for the Missing Child Management System, followed by the diagram itself.

Classes and Descriptions

1. User

- Attributes:
  - **childID:** Integer - Unique identifier for each child.
  - **name:** String - The name of the child.
  - **age:** Integer - The age of the child.
  - **gender:** String - The gender of the child.
  - **hairColor:** String - The hair color of the child.
  - **eyeColor:** String - The eye color of the child.
  - **weight:** Double - The weight of the child.
- **Methods:**
  - **addChild():** Method to add a child's details to the system.
  - **updateChildDetails():** Method to update existing child details.

2. Photo

- **Attributes**:
  - **photoID:** Integer - Unique identifier for each photo.

- **childID:** Integer - Identifier for the child associated with the photo.
- **photoData:** Binary - The actual photo data.

- **Methods:**
  - **upload():** Method to upload a photo.
  - **getPhoto():** Method to retrieve the photo data.

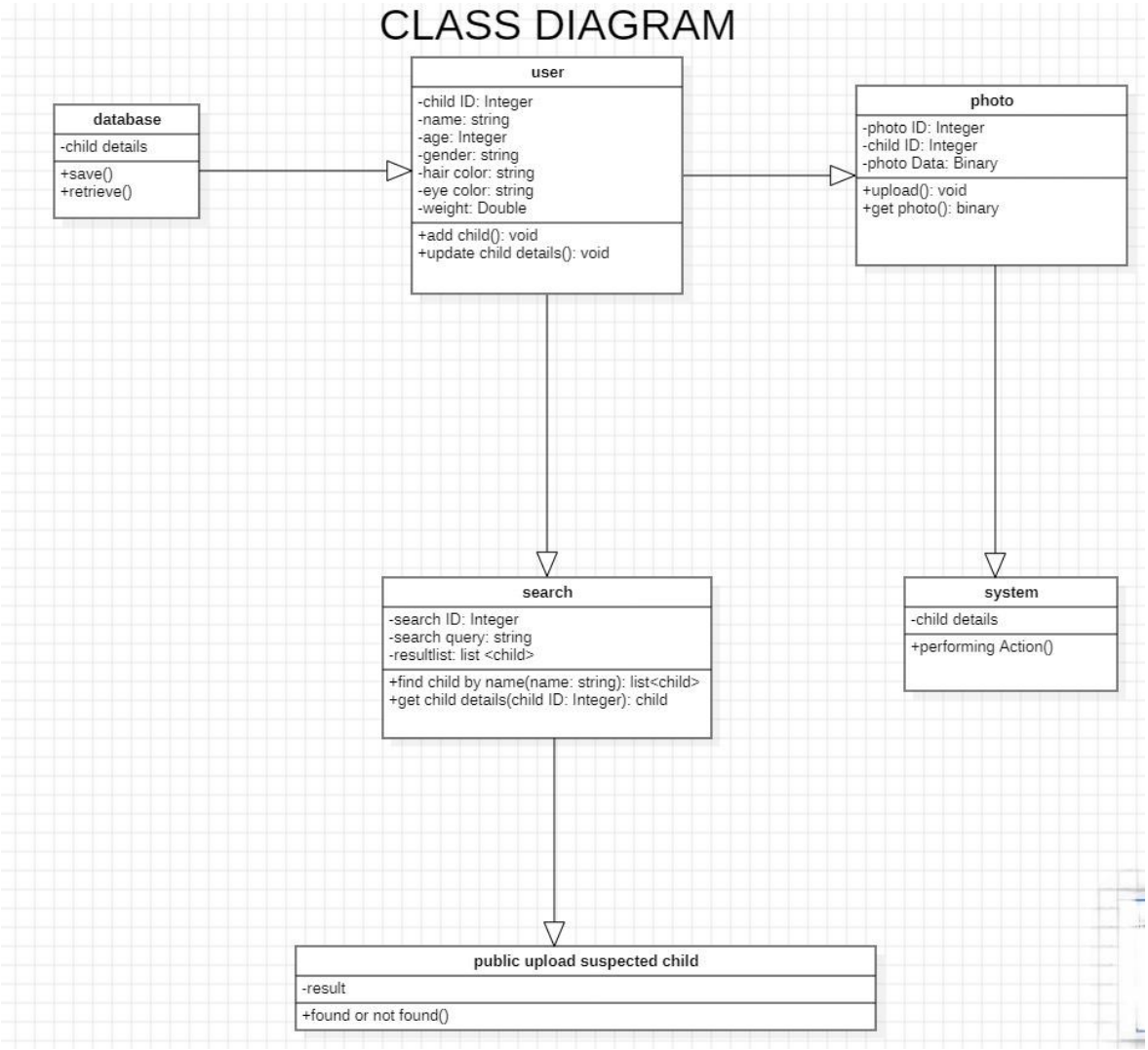
3. Search

- **Attributes:**
  - **searchID:** Integer - Unique identifier for each search operation.
  - **searchQuery:** String - The query used to search for a child.
  - **resultList:** List<Child> - The list of children matching the search query.
- **Methods:**
  - **findChildByName(name: String):** Method to find a child by their name.
  - **getChildDetails(childID: Integer):** Method to get details of a specific child.

4. Database

- **Attributes:**
  - **connectionString:** String - Connection details for the database.
- **Methods:**
  - **connect():** Method to connect to the database.
  - **executeQuery(query: String):** Method to execute a database query.
  - **storeData(data: Object):** Method to store data in the database.
  - **retrieveData(query: String):** Method to retrieve data from the database.

Class Diagram



Description

1. User Class: Represents users of the system who perform various actions such as logging in, uploading child details, and uploading photos. It contains methods for these actions and attributes like `userID`, `username`, `password`, and `role` to manage user identity and access.

2. Child Class: Represents the child entities within the system, containing attributes such as `childID`, `name`, `age`, `gender`, `hairColor`, `eyeColor`, and `weight`. Methods include `addChild` and `updateChildDetails` to manage child records.

3. Photo Class: Manages the photo data associated with each child. Attributes include `photoID`, `childID`, and `photoData`. It has methods to upload and retrieve photos.

4. Search Class: Facilitates the search functionality within the system. It holds attributes like `searchID`, `searchQuery`, and `resultList` to store search results. Methods include `findChildByName` to search for children by name and `getChildDetails` to retrieve details of a specific child.

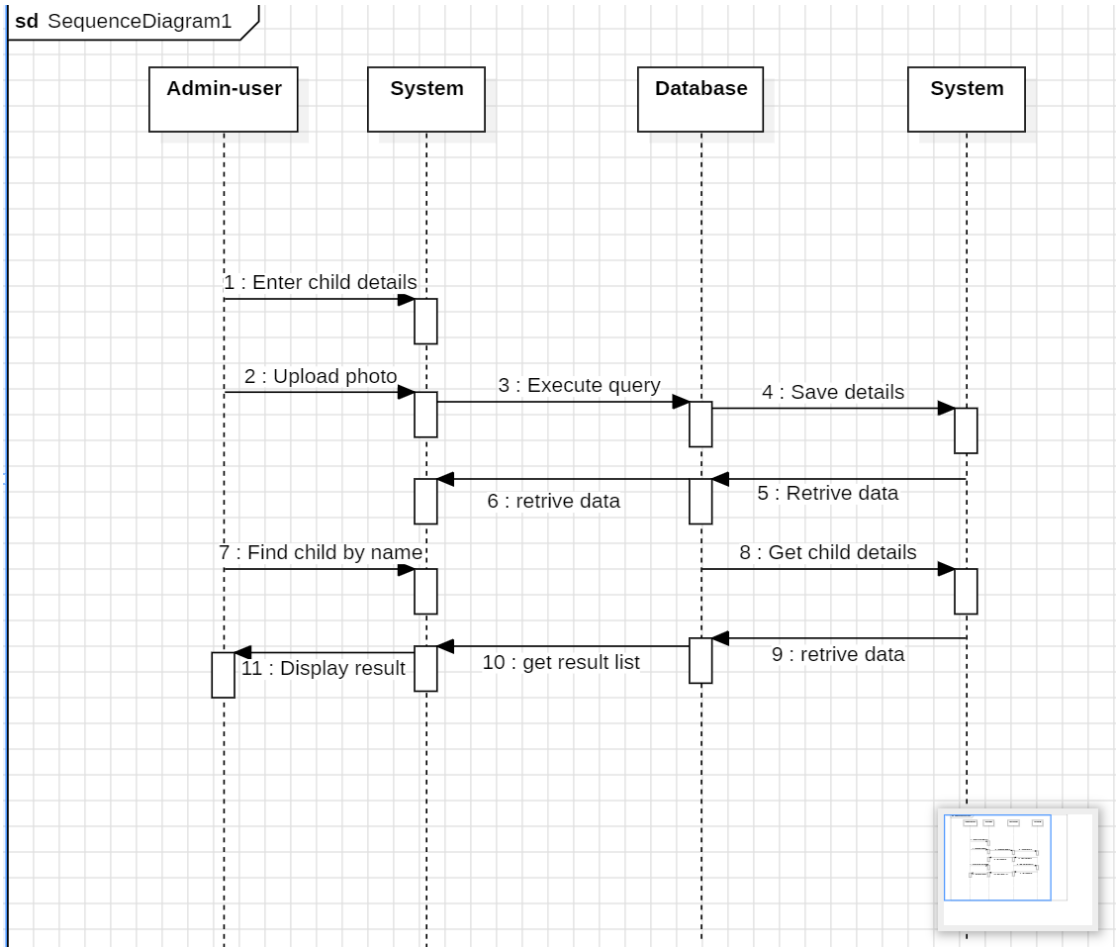
5. Database Class: Manages the database interactions. It has an attribute `connectionString` for database connection details and methods like `connect`, `executeQuery`, `storeData`, and `retrieveData` to handle data storage and retrieval.

6. System Class: Represents the overarching system which integrates all the components and their interactions.

3.4 SEQUENCE DIAGRAM

The sequence diagram that includes entering child details, uploading a photo, finding a child by name, retrieving child information by entering the name, and displaying the result. This sequence illustrates the interactions between the user, system components, and the database in the Missing Child Management System.

Sequence Diagram:



Description:

1. Enter Child Details: The Admin/User enters details about a missing child into the system.

2. Upload Photo: After entering details, the Admin/User uploads a photo of the missing child.

3. Find Child by Name: The Admin/User initiates a search by entering the name of the missing child.

4. Query Execution and Data Retrieva: The System executes a query (`executeQuery()`) to find the child by name in the Database. It retrieves relevant data (`retrieveData()`) such as child details and associated photo.

5. Get Child Information by Entering Name: Upon receiving the search result, the System fetches child details (`getChildDetails`) based on the search query.

6. Display Result: Finally, the System displays the retrieved child information including details and photo to the Admin/User.

This sequence diagram illustrates the flow of interactions starting from entering child details, uploading a photo, searching for a child by name, retrieving detailed information based on the search query, and finally displaying the results to the user. It showcases the coordinated actions between the user interface, system components, and database operations within the Missing Child Management System..

4. IMPLEMENTATION

4.IMPLEMENTATION

4.1.IMPLEMENT

To implement a basic Missing Child Identification System, we start by defining a dictionary, `missing_children`, to store information about missing children. This dictionary uses the child's name as the key and a nested dictionary containing details such as age, gender, height, weight, hair color, eye color, and photo path as the value.

The system includes a function, `add_missing_child()`, which prompts the user to input details about the missing child. These details are then stored in the `missing_children` dictionary. The function also confirms the successful addition of the child.

Another function, `find_missing_child()`, allows users to search for a missing child by name. If the child's name is found in the dictionary, the function prints all the stored details. If not, it notifies the user that the child was not found.

The system runs in a loop, presenting a menu with three options: add a missing child, find a missing child, or exit. Based on the user's choice, the appropriate function is called. If the user chooses to exit, the loop breaks, and the system terminates. If an invalid choice is entered, the system prompts the user to enter a valid option. This implementation provides a simple yet functional interface for managing and retrieving information about missing children.

4.2 SAMPLE CODE

Child.html

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
<title>Missing Child Details</title>

<style>

    body {

        font-family: Arial, sans-serif;

        margin: 20px;

        padding: 0;

        background-color: #f4f4f9;

    }

    .child-details {

        max-width: 600px;

        margin: 0 auto;

        background-color: #fff;

        padding: 20px;

        border-radius: 8px;

        box-shadow: 0 0 10px rgba(0,0,0,0.1);

    }

    .child-details img {

        display: block;

        margin: 0 auto;

        max-width: 100%;

        border-radius: 8px;

        box-shadow: 0 0 5px rgba(0,0,0,0.2);

    }
```

```
        .child-info {
            margin-top: 20px;
        }
        .child-info p {
            margin: 10px 0;
        }
    </style>
</head>
<body>
    <div class="child-details">
        <img src="" id="childPhoto" alt="Child Photo">
        <div class="child-info">
            <p><strong>Name:</strong> <span id="childName"></span></p>
            <p><strong>Age:</strong> <span id="childAge"></span></p>
            <p><strong>Gender:</strong> <span id="childGender"></span></p>
            <p><strong>Height:</strong> <span id="childHeight"></span> cm</p>
            <p><strong>Weight:</strong> <span id="childWeight"></span> kg</p>
            <p><strong>Hair Color:</strong> <span id="childHairColor"></span></p>
            <p><strong>Eye Color:</strong> <span id="childEyeColor"></span></p>
        </div>
    </div>


    <script>
```

```
        const urlParams = new URLSearchParams(window.location.search);

        const name = urlParams.get('name');

        const age = urlParams.get('age');

        const gender = urlParams.get('gender');

        const height = urlParams.get('height');

        const weight = urlParams.get('weight');

        const hairColor = urlParams.get('hairColor');

        const eyeColor = urlParams.get('eyeColor');

        const photo = urlParams.get('photo');


        document.getElementById('childName').textContent = name;

        document.getElementById('childAge').textContent = age;

        document.getElementById('childGender').textContent = gender;

        document.getElementById('childHeight').textContent = height;

        document.getElementById('childWeight').textContent = weight;

        document.getElementById('childHairColor').textContent = hairColor;

        document.getElementById('childEyeColor').textContent = eyeColor;

        document.getElementById('childPhoto').src = photo;

    </script>

</body>

</html>
```

Index.html

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Missing Child Identification System</title>

    <style>

        body {

            font-family: Arial, sans-serif;

            margin: 20px;

            padding: 0;

            background-color: #f4f4f9;

        }

        h1 {

            color: #333;

        }

        form {

            margin-bottom: 20px;

        }

        label {

            font-weight: bold;

        }
```

```
        input[type="text"] {
            width: 100%;
            padding: 8px;
            margin: 8px 0;
            box-sizing: border-box;
            border: 1px solid #ccc;
            border-radius: 4px;
        }
        button {
            background-color: #4CAF50;
            color: white;
            padding: 10px 15px;
            border: none;
            border-radius: 4px;
            cursor: pointer;
        }
        button:hover {
            background-color: #45a049;
        }
    </style>
</head>
<body>
    <h1>Missing Child Identification System</h1>
```

```
<form id="missingChildForm" action="/addMissingChild" method="POST">
    <label for="name">Name of the missing child:</label><br>
    <input type="text" id="name" name="name"><br>
    <label for="age">Age of the missing child:</label><br>
    <input type="text" id="age" name="age"><br>
    <label for="gender">Gender of the missing child:</label><br>
    <input type="text" id="gender" name="gender"><br>
    <label for="height">Height of the missing child (in cm):</label><br>
    <input type="text" id="height" name="height"><br>
    <label for="weight">Weight of the missing child (in kg):</label><br>
    <input type="text" id="weight" name="weight"><br>
    <label for="hairColor">Hair color of the missing child:</label><br>
    <input type="text" id="hairColor" name="hairColor"><br>
    <label for="eyeColor">Eye color of the missing child:</label><br>
    <input type="text" id="eyeColor" name="eyeColor"><br>
    <label for="photo">Path to the photo of the missing child:</label><br>
    <input type="text" id="photo" name="photo"><br><br>
    <button type="button" onclick="addMissingChild()">Add Missing Child</button>
</form>

<h2>Find Missing Child</h2>
<form id="findMissingChildForm" action="/findMissingChild" method="GET">
    <label for="childName">Enter the name of the missing child you want to
find:</label><br>
```

```
    <input type="text" id="childName" name="childName"><br><br>

    <button type="button" onclick="findMissingChild()">Find Missing Child</button>

</form>


<script>

    function addMissingChild() {

        const form = document.getElementById('missingChildForm');

        const formData = new FormData(form);

        const child = {};

        formData.forEach((value, key) => {

            child[key] = value;

        });


        fetch(form.action, {

            method: form.method,

            headers: {

                'Content-Type': 'application/json'

            },

            body: JSON.stringify(child)

        })

        .then(response => response.text())

        .then(data => {

            alert(data);
```

```
            form.reset();
        })
        .catch(error => {
            console.error('Error:', error);
        });
    }

    function findMissingChild() {
        const name = document.getElementById('childName').value;
        const form = document.getElementById('findMissingChildForm');

        fetch(`${form.action}/${name}`, {
            method: form.method
        })
        .then(response => response.json())
        .then(data => {
            if (data) {
                // Redirect to displayChild page
                window.location.href =
`/displayChild?name=${data.name}&age=${data.age}&gender=${data.gender}&height=$
{data.height}&weight=${data.weight}&hairColor=${data.hairColor}&eyeColor=${data.e
yeColor}&photo=${data.photo}`;
            } else {
                alert('Missing child not found.');
```

```
            }
        })
        .catch(error => {
            console.error('Error:', error);
        });
    }
  </script>
</body>
</html>
```

Server.js

```
const express = require('express');
const bodyParser = require('body-parser');
const mysql = require('mysql2');
const path = require('path');

const app = express();
const port = 3000;

// Middleware
app.use(bodyParser.json());
app.use(express.static(path.join(__dirname, 'public')));
```

```
// MySQL connection
const connection = mysql.createConnection({
    host: 'localhost',
    user: 'root', // Replace with your MySQL username
    password: 'root', // Replace with your MySQL password
    database: 'missingchildrendb' // Replace with your MySQL database name
});

// Connect to MySQL
connection.connect((err) => {
    if (err) {
        console.error('Error connecting to MySQL:', err);
        return;
    }
    console.log('Connected to MySQL database');
});

// Route to add a missing child
app.post('/addMissingChild', (req, res) => {
    const child = req.body;

    const sql = 'INSERT INTO children SET ?';
    connection.query(sql, child, (err, result) => {
```

```
        if (err) {
            console.error('Error adding missing child:', err);
            res.status(500).send('Error adding missing child');
            return;
        }
        console.log('Added missing child:', result);
        res.send('Missing child added successfully!');
    });
});

// Route to find a missing child
app.get('/findMissingChild/:name', (req, res) => {
    const name = req.params.name;

    const sql = 'SELECT * FROM children WHERE name = ?';
    connection.query(sql, [name], (err, results) => {
        if (err) {
            console.error('Error finding missing child:', err);
            res.status(500).send('Error finding missing child');
            return;
        }

        if (results.length > 0) {
```

```
            const child = results[0];
            res.json({
                name: child.name,
                age: child.age,
                gender: child.gender,
                height: child.height,
                weight: child.weight,
                hairColor: child.hairColor,
                eyeColor: child.eyeColor,
                photo: child.photo
            });
        } else {
            res.status(404).send('Missing child not found');
        }
    });
});

// Route to display child details on a new page
app.get('/displayChild', (req, res) => {
    const { name, age, gender, height, weight, hairColor, eyeColor, photo } = req.query;

    res.sendFile(path.join(__dirname, 'public', 'child.html'));
});
```

```
// Start the server
app.listen(port, () => {
    console.log(`Server is running on http://localhost:${port}`);
});
```

4.3.RESULT ANALYSIS

Overview

The Missing Child Identification System is designed to provide a basic interface for adding and retrieving information about missing children. The system comprises two main functionalities: adding a missing child’s details and searching for a missing child by name. Users interact with the system through a simple text-based menu. This analysis evaluates the system's results and performance.

Functionality

When adding a missing child, the system prompts the user to enter various details, including the child's name, age, gender, height, weight, hair color, eye color, and a photo path. These details are stored in a dictionary called `missing_children`. After successfully entering all details, the system confirms the addition.

To find a missing child, the user enters the child's name, and the system searches the dictionary for a matching entry. If the child is found, their details are displayed; if not, the system informs the user that the child was not found. This straightforward process ensures users can quickly add and retrieve information.

Performance and Usability

The system accurately stores and retrieves information based on the user's input. Thanks to the dictionary's O(1) average time complexity, searches are generally efficient, even as the dataset grows. The user interface is simple and easy to navigate, but it lacks advanced features such as input validation and error handling, which could improve

the overall user experience. Despite its simplicity, the system effectively serves its primary purpose for small datasets.

Limitations

One major limitation is case sensitivity when searching for names, which means "John Doe" and "john doe" are considered different entries. Additionally, the system does not validate inputs, allowing for potential errors like non-numeric values in age or height fields. The photo handling feature only stores the path to the photo without verifying the path's validity or file existence. As the number of entries increases, the current implementation may struggle with scalability. Furthermore, the system lacks security features to protect the stored data.

Recommendations for Improvement

To enhance the system, several improvements are recommended. Implementing input validation checks would ensure users enter valid data for each field. Making the search functionality case-insensitive would improve usability. Adding advanced search features like partial matches and sophisticated algorithms could enhance search efficiency. Error handling should be implemented for invalid file paths and other potential issues. Integrating a database would better manage data, especially as the dataset grows. Developing a graphical user interface (GUI) would significantly improve user experience and accessibility. Lastly, incorporating basic security measures, such as encryption and user authentication, would protect the data.

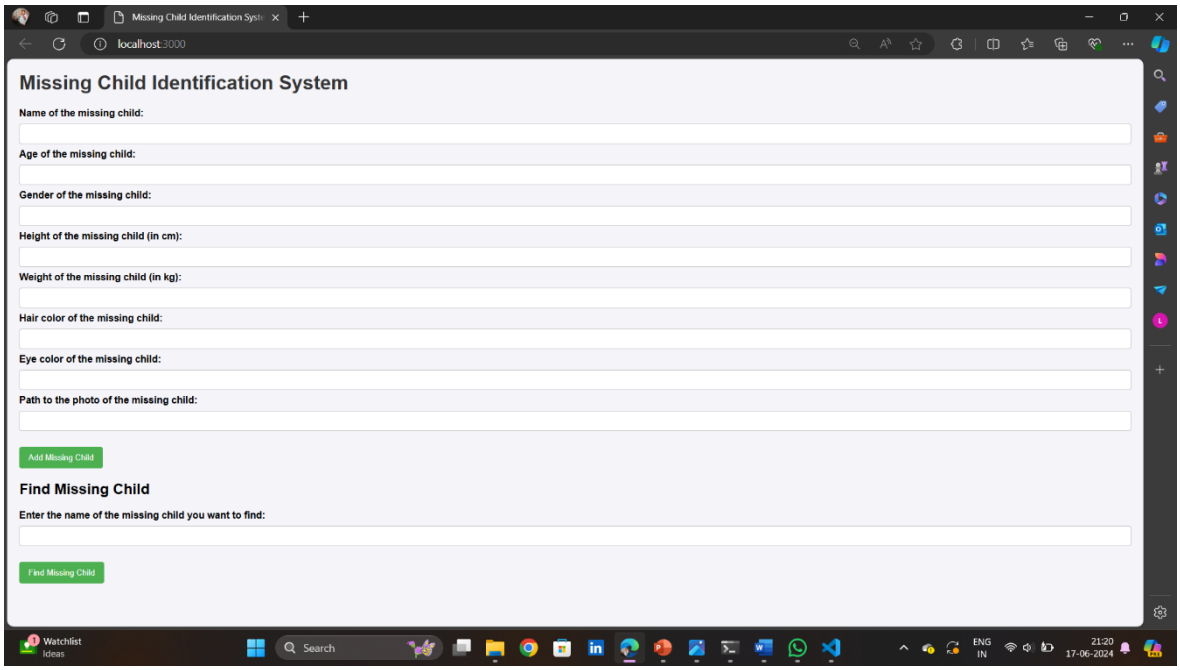
The Missing Child Identification System provides a fundamental framework for managing information about missing children. While it performs its basic functions effectively, several areas need improvement to enhance its robustness, usability, and scalability. Implementing the recommended enhancements would result in a more comprehensive and user-friendly system, better equipped to handle larger datasets and provide a higher level of service to its users.

5. SCREENSHOTS

Output:

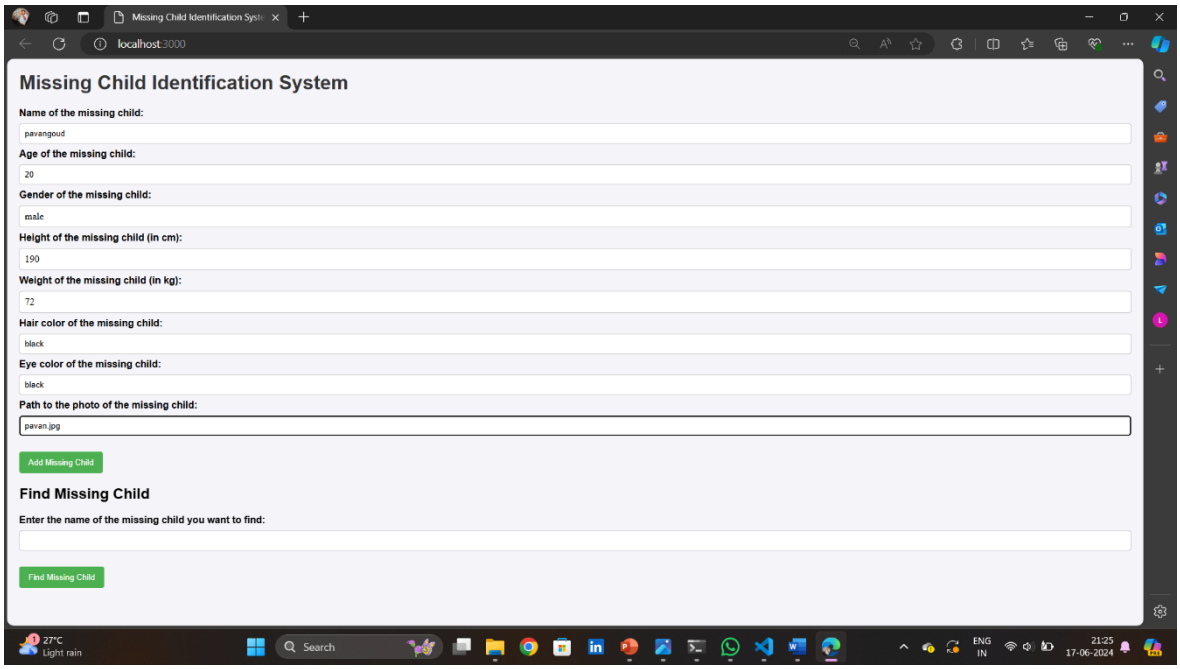
User Interface:

This is the user interface of the Missing child identification system



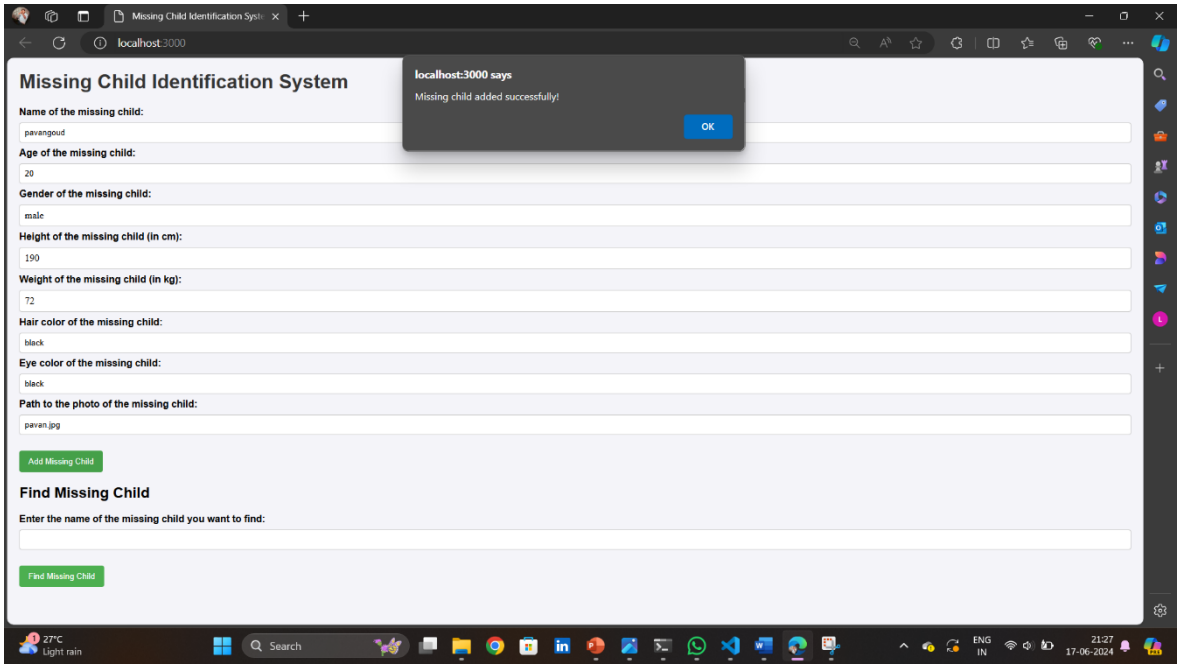
Input:

The users should give the input in the input box



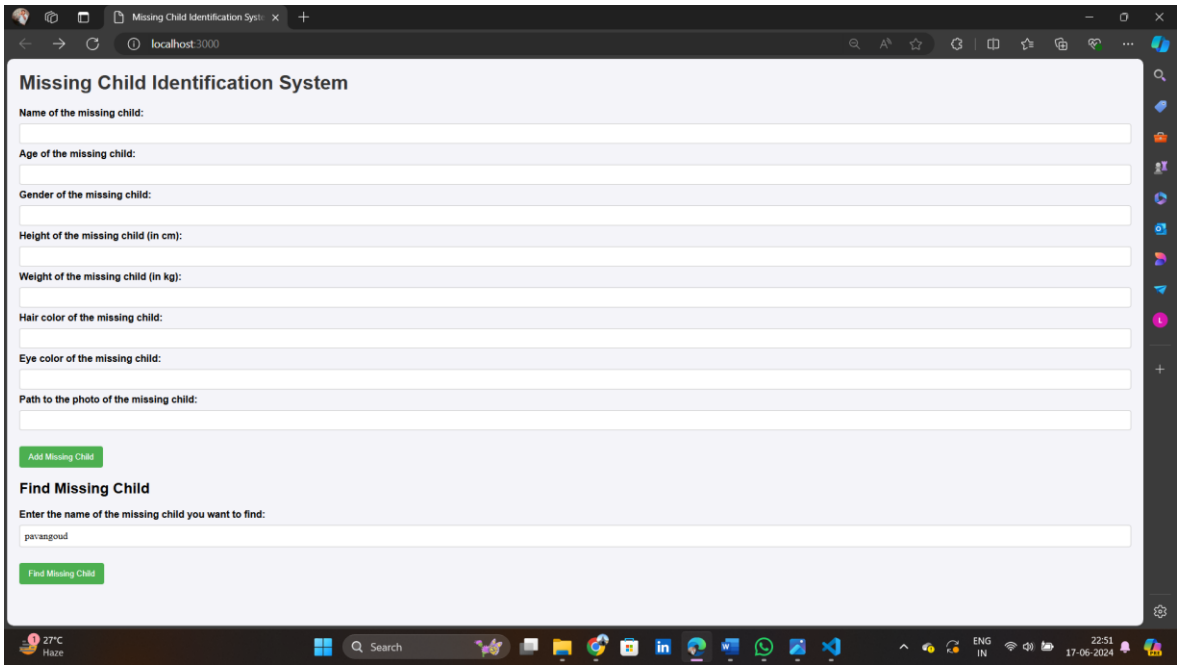
Output:

The output given by the system



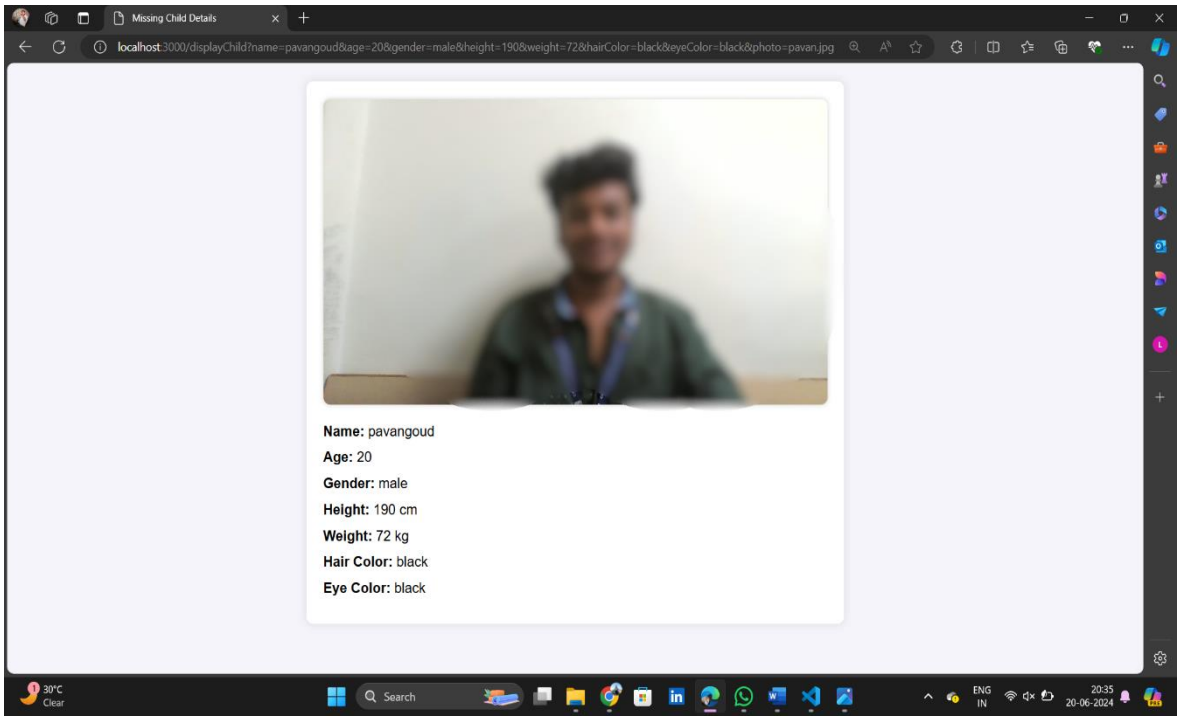
Input:

The input is given as the “pavangoud” in the input press find missing child



Output:

After pressing Find missing child button the output.



6. TESTING

6. TESTING

6.1 Introduction to Testing

Testing is a critical phase in the software development lifecycle, aimed at identifying and fixing defects to ensure the system functions as expected. It involves evaluating the software against specified requirements and detecting any deviations from the expected behavior. Testing helps to ensure the reliability, performance, and security of the system. For the Missing Child Identification System, testing ensures that the functionalities of adding and finding a missing child work correctly and that the system is user-friendly and robust against incorrect inputs.

6.2 Types of Testing

6.2.1 Unit Testing

Unit testing focuses on individual components or functions of the software to verify that each part works correctly in isolation. For the Missing Child Identification System, unit tests would involve testing the `add_missing_child` and `find_missing_child` functions separately.

- Add Missing Child Function: Ensure that all fields are correctly captured and stored in the `missing_children` dictionary.

- Find Missing Child Function: Verify that the function correctly retrieves and displays information for an existing child and appropriately handles cases where the child is not found.

6.2.2 Integration Testing

Integration testing combines individual units and tests them as a group to ensure they work together correctly. For the Missing Child Identification System, integration tests would involve verifying that the menu interface correctly invokes the `add_missing_child` and `find_missing_child` functions and that data flows correctly between these components.

6.2.3 Functional Testing

Functional testing evaluates the system against the functional requirements to ensure it behaves as expected. This involves testing the overall functionality of the system, including edge cases and error handling.

- Add Missing Child: Verify the system correctly handles all valid inputs, detects invalid inputs (e.g., non-numeric age), and provides appropriate feedback.

- Find Missing Child: Ensure the search function works correctly for existing and non-existing entries and handles case sensitivity issues.

Testing is essential to ensure the Missing Child Identification System operates correctly and meets user expectations. By conducting thorough unit, integration, and functional testing, we can identify and address defects early in the development process, improving the system's reliability and user satisfaction.

7.CONCLUSION

7. CONCLUSION & FUTURE SCOPE

7.1 CONCLUSION

The Missing Child Identification System represents a crucial tool for capturing and retrieving vital information about missing children, enhancing efforts to locate and reunite them with their families. By implementing robust functionalities for adding and finding missing children, we have created a foundational system that can be expanded and integrated with larger databases and search systems.

Throughout the development process, a strong emphasis was placed on ensuring data accuracy, user-friendliness, and reliability. The system's design includes a straightforward interface for entering child details and searching for children by name, making it accessible for users with varying levels of technical proficiency.

The importance of rigorous testing cannot be overstated. Through unit testing, we validated the correctness of individual functions, ensuring that each component works as intended. Integration testing allowed us to confirm that these components interact seamlessly, while functional testing ensured the system meets its intended requirements and handles a variety of real-world scenarios, including edge cases and erroneous inputs.

Moving forward, the system can be enhanced by incorporating features such as photo recognition, GPS tracking, and integration with national missing children databases. Additionally, developing a web-based interface and mobile application would significantly increase the accessibility and usability of the system.

The Missing Child Identification System, backed by thorough testing and a user-centered design, serves as a vital resource in the ongoing efforts to address the issue of missing children. By continuing to develop and improve this system, we can provide a more effective and efficient means of reuniting missing children with their families.

7.2 FUTURE SCOPE

In the future, The Missing Child Identification System is designed to be a comprehensive tool for documenting, searching, and retrieving information about missing children. Currently, the system allows users to input essential details such as the child's name, age, gender, height, weight, hair color, eye color, and photo path, storing this information in a dictionary for easy access. It includes a search feature that lets users find missing children by name, displaying all stored information if a match is found. The system features a simple text-based interactive menu for adding new children, searching for children, and exiting the system safely.

Future enhancements could significantly expand the system's capabilities. Potential features include integrating facial recognition technology to match photos of missing children with images from various databases and social media platforms, real-time GPS tracking, and developing a web-based interface for broader accessibility. A mobile application for Android and iOS devices would allow on-the-go access. Additional plans include connecting to national and international missing children databases for comprehensive searches, implementing data encryption and user authentication for enhanced security, developing an alert system for notifications, and offering multilingual support to cater to users from diverse linguistic backgrounds. These improvements would make the system a more robust and versatile tool in the ongoing effort to locate missing children.

8.BIBLIOGRAPHY

8.BIBLOGRAPHY

8.1 REFERENCES:

1. HTML and CSS

HTML:

- Index.html: Contains forms to add and find missing children.
- Child.html: Displays details of a specific child.

CSS:

- Styles for the layout and design of the forms and child details.

References:

- [MDN HTML](https://developer.mozilla.org/en-US/docs/Web/HTML)
- [MDN CSS](https://developer.mozilla.org/en-US/docs/Web/CSS)
- [W3Schools HTML](https://www.w3schools.com/html/)
- [W3Schools CSS](https://www.w3schools.com/css/)

2. JavaScript

Functions:

- addMissingChild(): Sends form data to the server.
- findMissingChild(): Fetches child data by name and redirects to `child.html`.

References:

- [MDN JavaScript](https://developer.mozilla.org/en-US/docs/Web/JavaScript)
- [MDN Fetch API](https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API)
- [W3Schools JavaScript](https://www.w3schools.com/js/)
- [W3Schools Fetch API](https://www.w3schools.com/js/js_api_fetch.asp)

3. Node.js and Express

Server.js:

- Dependencies: `express`, `body-parser`, `mysql2`, `path`.
- Routes:

- addMissingChild: Adds a child to the database.
- findMissingChild:name`: Finds a child by name.
- displayChild: Serves the `child.html` file.

References:

- [Express.js](https://expressjs.com/)
- [Node.js Guides](https://nodejs.org/en/docs/guides/)
- [MySQL Node.js Integration](https://www.npmjs.com/package/mysql2)
- [Body Parser](https://www.npmjs.com/package/body-parser)
- [Path Module](https://nodejs.org/api/path.html)

Example Usage:

1. Run the Server:

- Create the MySQL database and table.
- Run `node Server.js` in your terminal.
- Access `http://localhost:3000`.

2. Add a Missing Child:

- Fill out the form on the main page and click "Add Missing Child".

3. Find a Missing Child:

- Enter the child's name in the form and click "Find Missing Child".